

DAY 2 — Users, Groups, Permissions & Ownership

Learning Objectives

By the end of today, you will:

- Understand Linux's permission model (the #1 most-tested Linux topic in interviews)
- Create/manage users and groups like a real sysadmin
- Read and modify permissions with `chmod` and `chown` confidently
- Explain real production access-control scenarios

Daily Outcome

You can look at `ls -la` output, read the permission string instantly, and explain/fix access issues the way a Linux admin does on Day 1 of a job.

1. Why This Matters (WHY before HOW)

Production reality: Almost every "server issue" ticket in a real job is either a permission problem, a disk problem, or a service problem. Permissions specifically cause things like:

- "I can't SSH into the server" → often wrong key permissions
- "The application won't start" → wrong ownership on config/log files
- "Deploy script fails" → the deployment user doesn't have write access

Interview angle: Permission questions are asked in almost every single Linux/DevOps interview because they reveal whether you've actually worked hands-on or just read theory.

2. Users and Groups — Beginner to Advanced

Beginner concept: Linux is a multi-user OS. Every file/process belongs to a user and a group. This is how the OS decides who can do what.

Key files (industry-important, memorize these):

File	Purpose
<code>/etc/passwd</code>	List of all users (name, UID, home dir, shell)
<code>/etc/shadow</code>	Encrypted passwords (root-only readable)
<code>/etc/group</code>	List of all groups and their members

Intermediate concept — User types:

Type	Example	UID Range
Root	root	0
System users	www-data, sshd	1-999 (varies by distro)
Regular users	you, deploy, jenkins	1000+

Real production example: Companies never let developers deploy as `root`. Instead, they create a dedicated user like `deploy` or `jenkins` with only the permissions it needs. This is the **Principle of Least Privilege** — a term you should use in interviews.

Advanced/Industry concept: In real infra, user creation is automated via Ansible/Terraform, not done manually — but you must understand the manual commands first because that's what's tested in interviews and needed for debugging.

3. Permissions — The Core Model

Every file has 3 permission sets, for 3 categories of people:

```
-rwxr-xr-- 1 anuj devs 1024 Jul 5 10:00 deploy.sh
```

Breaking this down:

-	rwx	r-x	r--
type	owner	group	others

Symbol	Meaning
<code>r</code>	read (4)
<code>w</code>	write (2)
<code>x</code>	execute (1)
<code>-</code>	no permission

Numeric (octal) system — MUST know cold for interviews:

Permission	Value
read	4
write	2
execute	1

So `rwxr-xr--` = owner(4+2+1=7), group(4+0+1=5), others(4+0+0=4) → **754**

Analogy: Think of a file like an office document.

- Owner = you, the document's creator — full control
- Group = your team — limited access (e.g., read + open, not edit)
- Others = everyone else in the building — maybe can only view it

4. Special Permissions (Advanced — commonly asked)

Special Perm	Symbol	Use Case
SUID	<code>s</code> in owner's x slot	Run a program as the file owner (e.g., <code>passwd</code> command runs as root even when a regular user runs it)
SGID	<code>s</code> in group's x slot	New files in a directory inherit the group
Sticky Bit	<code>t</code> in others' x slot	In shared folders like <code>/tmp</code> , users can only delete their OWN files, not others'

Real production example: `/tmp` has the sticky bit set — that's why hundreds of users can dump files there but nobody can delete someone else's file.

5. Ownership

Every file has an **owner** (user) and a **group**. Two commands control this:

```
bash
chown anuj:devs deploy.sh
```

→ Changes owner to `anuj` and group to `devs`.

```
bash
chmod 754 deploy.sh
```

→ Sets permission to `rwxr-xr--` as calculated above.

```
bash
chmod u+x deploy.sh
```

→ Symbolic method: adds execute permission for the user (owner) only.

6. Commands — Line by Line

```
bash
sudo useradd -m -s /bin/bash deploy
```

→ `useradd` creates a user. `-m` creates a home directory. `-s /bin/bash` sets bash as default shell. Without `-m`, no home folder is created — a very common mistake.

```
bash
sudo passwd deploy
```

→ Sets a password for the `deploy` user.

```
bash
```

```
sudo groupadd devs
```

→ Creates a new group called `devs`.

```
bash
```

```
sudo usermod -aG devs deploy
```

→ `-aG` = **append to Group**. Adds `deploy` to the `devs` group WITHOUT removing them from other groups. (Forgetting `-a` and just using `-G` is a classic mistake — it overwrites all existing group memberships!)

```
bash
```

```
groups deploy
```

→ Shows which groups `deploy` belongs to.

```
bash
```

```
sudo chown anuj:devs myfile.txt
```

→ Changes both owner and group in one command.

```
bash
```

```
chmod 644 myfile.txt
```

→ Owner: read+write, Group: read, Others: read. Standard permission for regular files.

```
bash
```

```
chmod 755 myscript.sh
```

→ Owner: full control, Group/Others: read+execute. Standard permission for executable scripts.

7. Cheat Sheet

Command	Purpose
<code>useradd -m -s /bin/bash <user></code>	Create user with home dir
<code>passwd <user></code>	Set password
<code>groupadd <group></code>	Create group
<code>usermod -aG <group> <user></code>	Add user to group (safely)
<code>id <user></code>	Show UID, GID, groups
<code>chmod <perm> <file></code>	Change permissions
<code>chown <user>:<group> <file></code>	Change ownership
<code>chmod +x <file></code>	Make file executable

8. Real World Scenario

Scenario: A Jenkins CI/CD pipeline can't write logs to `/var/log/myapp/`. The error: `Permission denied`.

Troubleshooting steps (this is exactly how you'd answer in an interview):

1. `ls -la /var/log/myapp/` → check current owner/group/permissions
2. `whoami` (or check which user Jenkins runs as, e.g., `jenkins`)
3. If Jenkins user isn't the owner and isn't in the owning group → `sudo chown -R jenkins:jenkins /var/log/myapp/` or add jenkins to the appropriate group
4. Re-run the pipeline to confirm

This EXACT scenario has come up in real interviews as a "debug this" question.

9. Hands-On Lab

Steps:

1. Create a user: `sudo useradd -m -s /bin/bash testuser`
2. Set a password: `sudo passwd testuser`

3. Create a group: `sudo groupadd testgroup`
4. Add the user to the group: `sudo usermod -aG testgroup testuser`
5. Verify: `id testuser` — confirm the group shows up
6. Create a file: `touch myfile.txt`
7. Check default permissions: `ls -la myfile.txt`
8. Change ownership: `sudo chown testuser:testgroup myfile.txt`
9. Change permissions to 640: `chmod 640 myfile.txt`
10. Verify with `ls -la myfile.txt` — confirm it now reads `rw-r-----`

Expected Output (step 10):

```
-rw-r----- 1 testuser testgroup 0 Jul 5 10:15 myfile.txt
```

Common Errors:

Error	Cause	Fix
<code>useradd: Permission denied</code>	Not using sudo	Prefix with <code>sudo</code>
Group not showing in <code>id testuser</code>	Used <code>-G</code> instead of <code>-aG</code> , or forgot to re-login	Use <code>-aG</code> ; group changes may need a fresh shell session
<code>chown: invalid user</code>	Typo in username	Double check with <code>`cat /etc/passwd</code>

10. Mini Assignment

1. Create two users and one shared group. Add both users to the group.
2. Create a shared file both users should be able to read and write, but outsiders should not access at all. What `chmod` value did you use and why?

11. Quiz (10 Questions)

1. What does the `(r)`, `(w)`, `(x)` in a permission string mean?
2. What octal value represents `rwxr-xr--`?
3. What's the difference between `-G` and `-aG` in `usermod`?

4. Which file stores encrypted passwords?
 5. What is the Principle of Least Privilege?
 6. What does the sticky bit do, and where is it commonly used?
 7. What command changes both owner and group at once?
 8. What happens if you run `useradd` without `-m`?
 9. What permission would you give an executable script owned by you, readable by your team, and hidden from everyone else?
 10. Why don't production systems let apps run as root?
-

12. Interview Preparation

Q1: Explain Linux file permissions to someone who's never seen Linux.

| Expected: Use the office document analogy — owner/group/others, read/write/execute.

Q2 (Scenario): A script fails with "Permission denied" when executed. What do you check first?

| Expected: `ls -la script.sh` to check execute permission → `chmod +x script.sh` if missing.

Q3 (Cross-question): "What if `chmod +x` still doesn't fix it?"

| Expected: Check the shebang line (`#!/bin/bash`), check if the filesystem is mounted with `noexec`, check ownership.

Q4: What's the difference between SUID and SGID?

| Expected: SUID runs as file owner; SGID makes new files inherit the directory's group.

13. English Communication

10 Technical Words: permission, ownership, privilege, executable, credential, authentication, authorization, sudo, superuser, umask

10 Business Words: compliance, audit, access control, accountability, policy, governance, risk, mitigation, onboarding, offboarding

Speaking Task: Explain out loud, in under 60 seconds: "What is the Principle of Least Privilege, and why does it matter in a company?" Focus on pronouncing "privilege" and "authorization" clearly — these are commonly mispronounced.

Writing Task: Write a Slack message to a (fictional) teammate explaining that you fixed a permission issue on the server today. Keep it professional, 2-3 sentences. I'll correct it.

Daily Summary

- Learned the Linux permission model (rwx, octal notation)
- Created users and groups, managed group membership safely
- Learned special permissions (SUID/SGID/Sticky bit)
- Practiced a real production troubleshooting scenario

Revision Checklist

- ☐ Can convert any `(rwx)` string to octal in your head
- ☐ Know the difference between `(-G)` and `(-aG)`
- ☐ Completed the hands-on lab
- ☐ Attempted all 10 quiz questions

Homework

- Finish the Mini Assignment
- Practice converting 5 random permission strings to octal (e.g., `(rw-r--r--)`, `(rwxrwxrwx)`)
- Submit your Slack message writing task

Tomorrow Preview

Day 3: Special Permissions Deep Dive + ACLs — you'll learn Access Control Lists for when standard owner/group/others isn't enough (a real enterprise requirement).

Motivational Note

Permissions trip up even people with a few months of experience — you're building a foundation most people skip. Keep going. 🗝️🚀

This time, please share your actual lab output (the `(ls -la)` results) and quiz answers before we start Day 3 — it only takes 2 minutes and it's how I catch real mistakes early.